BRAC UNIVERSITY
Department of Computer Science and Engineering

Examination: Quiz 2 Duration: 30 min
Semester: Spring 2025 Full Marks: 15

CSE 470: Software Engineering

Name: **ID:** **Section:**

A hospital monitoring system needs to maintain a **centralized** vital signs dashboard that receives data from **multiple patient monitors** throughout the facility. The dashboard must **maintain a single source of truth** for all vital signs data and **notify** all connected nurse stations, doctor's tablets, and emergency response teams in **real-time** when any patient's vitals change.

Nurse stations and doctor's tablets will only display the information, but emergency response team monitors will send an emergency response.

The system must ensure that all medical staff see the same data **simultaneously** and that **there aren't multiple instances** of the dashboard that could lead to **inconsistent** patient monitoring.

Python

```
if __name__ == "__main__":
    # Central monitoring dashboard
    dash = Dashboard()

    # Create monitors
    nurse = Monitor("Nurse Station")
    doctor = Monitor("Doctor Tablet")

    # Subscribe monitors
    dash.subscribe(nurse)
    dash.subscribe(doctor)

    # Update vitals
    dash.update(room="101", hr=75, bp="120/80", o2=98)
    # Notifies both monitors

    # Add emergency team
    emt = Monitor("Emergency Team")
    dash.subscribe(emt)

    # Update with critical vitals
    dash.update(room="101", hr=120, bp="160/95", o2=92)
    # Notifies all three monitors
```

Java

```
public class Main {
    public static void main(String[] args) {
        // Central monitoring dashboard
        Dashboard dash = Dashboard.getInstance();

        // Create monitors
        Monitor nurse = new Monitor("Nurse Station");
        Monitor doctor = new Monitor("Doctor Tablet");

        // Subscribe monitors
        dash.subscribe(nurse);
        dash.subscribe(doctor);

        // Update vitals
        dash.update("101", 75, "120/80", 98);
        // Notifies both monitors

        // Add emergency team
        Monitor emt = new Monitor("Emergency Team");
        dash.subscribe(emt);

        // Update with critical vitals
        dash.update("101", 120, "160/95", 92);
        // Notifies all three monitors
    }
}
```

1.	**CO3**	a. Considering the above scenario, choose appropriate design pattern(s). Justify your answer. [4 Marks] b. Write the code necessary to complete the provided driver code by implementing your chosen design pattern(s). (Write only the classes you will add, **no need to write the main method**. You must implement the method names as given in the scenario.) [8 Marks] c. What are the different categories of Design Patterns? Name a design pattern in each category. [3 Marks]	**15**

a. The appropriate design patterns are Singleton and Observer pattern.
Singleton because we need
- a centralized vital signs dashboard **(central)**
- which must maintain a single source of truth **(consistency is needed)**
- and the system must ensure there aren't multiple instances of the dashboard **(single instance)**

Observer because we need
- the single dashboard (subject) to notify all connected nurse stations, etc (observers) **(one to many notifications)**
- they must be notified in real-time simultaneously **(automatic notifications)**

b. Minimum solution (just the notification functionality):

```
class Dashboard:
  _instance = None
   def __new__(cls)
      if _instance is None:
            cls._instance = super(Ledger, cls).__new__(cls)
            cls._instance.subscribers = []
       return cls._instance

  def subscribe(self, monitor):
     self.subscribers.append(monitor)

  def unsubscribe(self, monitor):
     self.subscribers.remove(monitor)

 def update(self, room, hr, bp, o2):
      self.notify(room, hr, bp, o2)

 def notify(self, room, hr, bp, o2):
    for s in subscribers:
         s.update_data(room, hr, bp, o2)

class Monitor:
  def update_data(self, room, hr, bp, o2):
        print()

class GenericMonitor(Monitor):  // Doctor and Nurse's
  def update_data(self, room, hr, bp, o2):
        print()

class EmergencyMonitor(Monitor): // Emergency Response Team
   def update_data(self, room, hr, bp, o2):
        print()
```

Most correct solution, as there are multiple rooms in which the vital data might be updated

```
class Dashboard:
   _instance = None
    def __new__(cls)
       if  _instance is None:
             cls._instance = super(Dashboard, cls).__new__(cls)
             cls._instance.subscribers = []
             cls._instance.room_vitals = {}
        return cls._instance

   def subscribe(self, monitor):
      self.subscribers.append(monitor)

   def unsubscribe(self, monitor):
      self.subscribers.remove(monitor)

  def update(self, room, hr, bp, o2):
       self.room_vitals[room] = (hr, bp, o2)
       self.notify(room)

  def notify(self, room):
     for s in subscribers:
          s.update_data(room, room[0], room[1], room[2])

class Monitor:
   def update_data(self, room, hr, bp, o2):
         print()

class GenericMonitor(Monitor):  // Doctor and Nurse's
   def update_data(self, room, hr, bp, o2):
         print()

class EmergencyMonitor(Monitor): // Emergency Response Team
    def update_data(self, room, hr, bp, o2):
         print()
```

c. The following are the three categories and examples
   i. Creational - Singleton
   ii. Behavioral - Observer
   iii. Structural - Adapter